V

Graphics

Some basic graphical concepts concerning pixels (‘picture elements’) are presented in Chapter 10. These are used to specify the raster-op function, an important graphical operation, in Chapter 11, and also a number of window systems later in the book.

Chapter 10

Basic Graphical Concepts

This chapter gives a formal framework to aid the description of pixels*, their organization into* pixel maps *and a number of* windows*. These definitions are used subsequently in Chapter 11 and Part VI.*

10.1 Background

The interest in formalizing aspects of computer graphics and interactive systems is gradually increasing [132]. For example, parts of the ISO/IEC GKS graphics standard have been formalized using Z [12, 13]. Early attempts have demonstrated the necessity of chosing an appropriate notation for the job [175]. Some formal notations have been designed with computer processing in mind [114] and readability can be a secondary consideration. However, in the case of standards, the latter becomes a much more important factor than the former.

The *Human-Computer Interface* (HCI) is an important part of most software systems, and window systems are currently the standard interface for the vast majority of workstations. Formalizing an HCI in a realistic and useful manner is a difficult task, but progress is being made in categorizing features of interfaces that may help to ensure their reliability in the future [128, 193]. There seems to be considerable scope for further work in this area.

10.2 Pixels

10.2.1 Pixel positions

A raster graphics display is made up of a set of pixels with positions or coordinates. These are normally defined in X–Y coordinate space. The display is a fixed size bounded rectangle in the X–Y plane.

$$Xsize, Ysize : \mathbb{N}_1$$

The offset in a particular direction is specified from zero up by convention. The position of a pixel may be specified by a pair of X–Y coordinates.

$$Xrange == 0 \,..\, Xsize - 1$$
$$Yrange == 0 \,..\, Ysize - 1$$
$$Pixel == Xrange \times Yrange$$

The pixel at $(0, 0)$ is normally at the upper left-hand corner of the display and the pixel at $(Xsize - 1, Ysize - 1)$ is at the bottom right by convention on most graphical computer-based window systems.

Many operations are applied to pairs of pixels.

$$
\begin{array}{|l}
\textit{PixelPair} \\
\hline
pix_1, pix_2 : Pixel \\
x_1, x_2 : Xrange \\
y_1, y_2 : Yrange \\
\hline
pix_1 = (x_1, y_1) \\
pix_2 = (x_2, y_2) \\
\hline
\end{array}
$$

The '$+$', '$-$' and '$\leq$' operators may be overloaded to apply to pixel positions. '$+$' and '$-$' may be used for moving pixel areas around the display. '$\leq$' can be used to define pixel ordering from the top left to bottom right.

$$
\begin{array}{|l}
_+_, \\
_-_ : (Pixel \times Pixel) \pfun Pixel \\
\leq : Pixel \rel Pixel \\
\hline
\forall\, PixelPair \bullet \\
\quad (x_1 + x_2 < Xsize \land y_1 + y_2 < Ysize) \Rightarrow \\
\qquad pix_1 + pix_2 = (x_1 + x_2, y_1 + y_2) \land \\
\quad (x_2 \leq x_1 \land y_2 \leq y_1) \Rightarrow \\
\qquad pix_1 - pix_2 = (x_1 - x_2, y_1 - y_2) \land \\
\quad pix_1 \leq pix_2 \Leftrightarrow x_1 \leq x_2 \land y_1 \leq y_2
\end{array}
$$

We can define the offset between any two pixel positions as a pixel offset. This is defined to wrap round the edge of the pixel area and thus is a total function.

$$
\begin{array}{|l}
offset : Pixel \to Pixel \tfun Pixel \\
\hline
\forall\, PixelPair \bullet \\
\quad offset\ pix_1\ pix_2 = ((x_1 + x_2) \bmod Xsize, (y_1 + y_2) \bmod Ysize)
\end{array}
$$

We can also overload the '. .' operator to define a rectangular area of pixels.

$$
\begin{array}{|l}
_._ : (Pixel \times Pixel) \to \mathbb{F}\, Pixel \\
\hline
\forall\, PixelPair \bullet \\
\quad pix_1 . . pix_2 = (x_1 \,..\, x_2) \times (y_1 \,..\, y_2)
\end{array}
$$

$$pix_1, pix_2 : Pixel \vdash pix_1 . . pix_2 = \{p : Pixel \mid pix_1 \leq p \land p \leq pix_2\}$$

A rectangular area of pixels can defined using any two opposing corners (e.g., returned using an attached mouse to sweep between the two). The following functions return the upper left and lower right pixel positions from two such pixel positions respectively.

$$
\begin{array}{|l}
_ \underline{min} _, \\
_ \underline{max} _ : (Pixel \times Pixel) \rightarrow Pixel \\
\hline
\forall\, PixelPair \bullet \\
\quad pix_1 \underline{min}\, pix_2 = (min\{x_1, x_2\}, min\{y_1, y_2\}) \land \\
\quad pix_1 \underline{max}\, pix_2 = (max\{x_1, x_2\}, max\{y_1, y_2\})
\end{array}
$$

10.2.2 Pixel maps

A raster graphics display has a number of bit-planes. This may be considered as the Z direction of the display.

$$
\begin{array}{|l}
Zsize : \mathbb{N}_1
\end{array}
$$

Each *bit* in a bit-plane has one of two values (cleared or set).

$$ClearVal == 0$$
$$SetVal == 1$$

$$BitVal == \{ClearVal, SetVal\}$$

The value of a pixel at a particular position may be modelled as a function from bit-plane number to bit value.

$$Zrange == 0 \,..\, Zsize - 1$$

$$Value == Zrange \rightarrow BitVal$$

If all the bits are clear the '*Value*' is considered '*Black*' and if they are all set it is considered '*White*'.

$$Black == (\, \mu\, val : Value \mid \operatorname{ran} val = \{ClearVal\}\,)$$

$$White == (\, \mu\, val : Value \mid \operatorname{ran} val = \{SetVal\}\,)$$

Note that if there is only one bit-plane (i.e., $Zsize = 1$) then pixel values can only be *Black* or *White*.

$$\vdash Zsize = 1 \Rightarrow Value = \{Black, White\}$$

A pixel map consists of a (partial) function from pixel positions to the value of the pixel contents. This can be used to describe part of a display, such as a window.

$$Pixmap == Pixel \pfun Value$$

Non-empty pixel maps may be of special interest.

$$Pixmap_1 == Pixmap \setminus \{\emptyset\}$$

Pixel maps are often rectangular in area. We can define such pixel maps using their bottom left and top right pixel positions.

$$Rectangle == \{map : Pixmap_1 \mid \exists_1 p_1, p_2 : Pixel \bullet \operatorname{dom} map = p_1 \,..\, p_2\}$$

Sometimes it is desirable to set all the range of a pixel map to a particular value, for

example when clearing a window down to the background colour. A function to set the range of a relation to a particular value is useful for this.

$$
\begin{array}{|l}
[P, V] \\
\hline
\mathit{setval} : V \to (P \pfun V) \to (P \pfun V) \\
\hline
\forall v : V;\ p : P \pfun V \bullet \\
\quad \mathit{setval}\ v\ p = (\ \mu\, m : P \pfun V \mid (\operatorname{dom} m = \operatorname{dom} p \land \operatorname{ran} m = \{v\})\ ) \\
\hline
\end{array}
$$

The following laws apply:

$$
\begin{array}{rcl}
p : \mathit{Pixmap};\ v : \mathit{Value} & \vdash & (\mathit{setval}\ v)^+\ p = \mathit{setval}\ v\ p \\
v : \mathit{Value} & \vdash & \mathit{setval}\ v\ \emptyset[\mathit{Pixel} \times \mathit{Value}] = \emptyset
\end{array}
$$

Two pixel maps may overlap. For example, one window may obscured by another. This can be captured as a relation between pixel maps:

$$
\begin{array}{|l}
[P, V] \\
\hline
_\ \underline{\mathit{overlaps}}\ _ : (P \pfun V) \leftrightarrow (P \pfun V) \\
\hline
\forall p_1, p_2 : P \pfun V \bullet \\
\quad p_1\ \underline{\mathit{overlaps}}\ p_2 \Leftrightarrow \operatorname{dom} p_1 \cap \operatorname{dom} p_2 \neq \emptyset \\
\hline
\end{array}
$$

A sequence of pixel maps may be overlaid in the order given by the sequence. It is convenient to define a distributed overriding operator for this.

$$
\begin{array}{|l}
[P, V] \\
\hline
\oplus/ : \operatorname{seq}(P \pfun V) \to (P \pfun V) \\
\hline
\oplus/\langle\rangle = \emptyset \\
\forall p : P \pfun V \bullet \oplus/\langle p \rangle = p \\
\forall s, t : \operatorname{seq}(P \pfun V) \bullet \oplus/(s \frown t) = (\oplus/s) \oplus (\oplus/t) \\
\hline
\end{array}
$$

Distributed overriding is particularly useful for defining the view on a screen of a display, given a sequence of possibly overlapping pixel maps.

The following laws apply for the distributed overriding operator:

$$
\begin{array}{rcl}
p_1, p_2 : \mathit{Pixmap} & \vdash & \oplus/\langle p_1, p_2 \rangle = p_1 \oplus p_2 \\
p : \mathit{Pixmap};\ s : \operatorname{seq} \mathit{Pixmap} & \vdash & \oplus/(s \frown \langle p \rangle) = (\oplus/s) \oplus p \\
p : \mathit{Pixmap};\ s : \operatorname{seq} \mathit{Pixmap} & \vdash & \oplus/(\langle p \rangle \frown s) = p \oplus (\oplus/s) \\
s : \operatorname{seq}_1 \mathit{Pixmap} & \vdash & \oplus/s = (\oplus/(\mathit{front}\ s)) \oplus (\mathit{last}\ s) \\
 & & \quad = (\mathit{head}\ s) \oplus (\oplus/(\mathit{tail}\ s)) \\
s : \operatorname{seq} \mathit{Pixmap} & \vdash & \operatorname{dom}(\oplus/s) = \operatorname{dom}(\bigcup(\operatorname{ran} s)) \\
s : \operatorname{seq} \mathit{Pixmap} & \vdash & \operatorname{ran}(\oplus/s) \subseteq \operatorname{ran}(\bigcup(\operatorname{ran} s)) \\
s : \operatorname{seq} \mathit{Pixmap} & \vdash & \bigcap(\operatorname{ran} s) = \emptyset \Rightarrow \oplus/s = \bigcup(\operatorname{ran} s)
\end{array}
$$

10.3 Windows

A series of windows on a display screen may be viewed as a sequence in which each window is laid on the screen in the order defined by the sequence (bottom first, top last). If some of the windows are removed from a sequence, it is sometimes desirable to 'squash' the remaining windows into a sequence again, keeping the windows in the same order, but ensuring that they are numbered sequentially from one upwards. The standard Z toolkit *squash* function (see page 118 of [381]) can be used to perform this operation.

If the windows in a sequence overlap, it is useful to be able to move selected windows so that their contents may be viewed (or hidden). This is analogous to shuffling a pile of sheets of paper (windows) on a desk (screen). Note that the sheets of paper may be of different sizes and in different positions on the desk.

We can define functions to 'select' and 'remove' a set of windows from a sequence using their identifiers rather than their position in the pile.

$$
\begin{array}{|l}
[W] \\
_ \underline{select} _, \\
_ \underline{remove} _ : \operatorname{seq} W \times \mathbb{P}\, W \rightarrow \operatorname{seq} W \\
\hline
\forall s : \operatorname{seq} W;\ w : \mathbb{P}\, W \bullet \\
\quad s \ \underline{select}\ w = squash(s \rhd w) \land \\
\quad s \ \underline{remove}\ w = squash(s \rhd w) \\
\hline
\end{array}
$$

We can then 'raise' or 'lower' these windows to the end or beginning of the sequence as required.

$$
\begin{array}{|l}
[W] \\
_ \underline{raise} _, \\
_ \underline{lower} _ : \operatorname{seq} W \times \mathbb{P}\, W \rightarrow \operatorname{seq} W \\
\hline
\forall s : \operatorname{seq} W;\ w : \mathbb{P}\, W \bullet \\
\quad s \ \underline{raise}\ w = (s \ \underline{remove}\ w) \frown (s \ \underline{select}\ w) \land \\
\quad s \ \underline{lower}\ w = (s \ \underline{select}\ w) \frown (s \ \underline{remove}\ w) \\
\hline
\end{array}
$$

Every window in a system usually has an identifier, denoted '*Window*', which allows it to be accessed uniquely.

$$[Window]$$

The generic functions defined in this section will normally be applied to such identifiers.

Windows often contain text. Thus, a text string is needed sometimes (e.g., for a title of a window). This is denoted as '*String*'. The string may be empty.

$$[String]$$

$$\mid \ `' : String$$

This concludes the basic definitions which will be used as required in Chapter 11 and Part VI.

Chapter 11

Raster-Op Functions

Raster-op *functions are useful when moving areas of pixels (e.g., parts of windows) around the screen on graphics display systems. Raster-op is now widely used in graphics systems, in particular for window systems. This chapter formally specifies raster-op functions and gives an example of its use. It makes use of some of the basic graphic concepts formalized in Chapter 10. Readers not familiar with 'raster-op' may prefer to do some background reading first (e.g., see [158, 312]).*

11.1 Pixel Operations

11.1.1 Pixel values

Given two pixel values, we may perform a bit-wise '*NAND*' ('not and') on the two values to produce a new value. If both bits are set then the result is clear; if either bit is clear then the result is set.

$$
\begin{array}{|l}
_ \underline{NAND} _ : (Value \times Value) \rightarrow Value \\
\hline
\forall\, val_1, val_2, val : Value \bullet \\
\quad val_1 \ \underline{NAND}\ val_2 = val \Leftrightarrow \\
\quad\quad (\forall\, n : Zrange \bullet \\
\quad\quad\quad (val_1\ n = SetVal \land val_2\ n = SetVal) \Rightarrow val\ n = ClearVal \land \\
\quad\quad\quad (val_1\ n = ClearVal \lor val_2\ n = ClearVal) \Rightarrow val\ n = SetVal\,)
\end{array}
$$

All the other binary and unary logical functions may be defined in terms of this function. For example, we can define a unary '*NOT*' function.

$$
\begin{array}{|l}
NOT : Value \rightarrow Value \\
\hline
\forall\, val : Value \bullet \\
\quad NOT\ val = val\ \underline{NAND}\ val
\end{array}
$$

We can also define binary '*AND*', '*OR*' and '*XOR*' functions.

$$
\begin{array}{|l}
_ \underline{AND} _, \\
_ \underline{OR} _, \\
_ \underline{XOR} _ : (Value \times Value) \rightarrow Value \\
\hline
\forall\, val_1, val_2 : Value \bullet \\
\quad val_1 \underline{AND}\, val_2 = NOT(val_1 \underline{NAND}\, val_2) \land \\
\quad val_1 \underline{OR}\, val_2 = (NOT\ val_1) \underline{NAND}\, (NOT\ val_2) \land \\
\quad val_1 \underline{XOR}\, val_2 = (val_1 \underline{OR}\, val_2) \underline{AND}\, (val_1 \underline{NAND}\, val_2)
\end{array}
$$

11.1.2 Pixel maps

A pixel map is considered to have the same shape as another if it can be 'moved' using a unique one-to-one function ($(\mathit{offset}\ pix_0)$ in the definition below) to give it the same domain. Intuitively this implies that each of the pixel maps could be moved *en masse* about the domain space so that its domain is exactly the same as the other map.

$$
\begin{array}{|l}
_ \underline{sameshape} _ : Pixmap \leftrightarrow Pixmap \\
\hline
\forall\, map_1, map_2 : Pixmap \bullet \\
\quad map_1 \underline{sameshape}\, map_2 \Leftrightarrow \\
\qquad (\exists\, pix_0 : Pixel \bullet \\
\qquad\quad \mathrm{dom}((\mathit{offset}\ pix_0) \fatsemi map_1) = \mathrm{dom}\, map_2\ )
\end{array}
$$

Consider a pair of (possibly overlapping) pixel maps which have the same shape.

$$
\begin{array}{|l}
\textit{MapPair} \\
\hline
map_1, \\
map_2 : Pixmap \\
\hline
map_1 \underline{sameshape}\, map_2 \\
\hline
\end{array}
$$

Two pixels in a pair of pixel maps with the same shape are considered to have the same position if the same offset function which relates the two maps also relates the two pixels. This is captured in '*samepos*' relation below. If the relation is true then it implies that the two pixels are in the same relative position within two pixel maps with the same shape. That is to say, if one of the pixel maps were to be moved on top of the other then one pixel would be exactly on top of the other.

$$
\begin{array}{|l}
_ \underline{samepos} _ : (Pixel \times Pixmap) \leftrightarrow (Pixel \times Pixmap) \\
\hline
\forall\, pix_1, pix_2 : Pixel;\ MapPair \mid \\
\quad pix_1 \in \mathrm{dom}\, map_1 \land pix_2 \in \mathrm{dom}\, map_2 \bullet \\
\qquad (pix_1, map_1) \underline{samepos}\, (pix_2, map_2) \Leftrightarrow \\
\qquad\quad (\exists\, pix_0 : Pixel \bullet \\
\qquad\qquad \mathrm{dom}((\mathit{offset}\ pix_0) \fatsemi map_1) = \mathrm{dom}\, map_2 \land \\
\qquad\qquad pix_1 \mapsto pix_2 \in \mathit{offset}\ pix_0\ )
\end{array}
$$

We may define operations similar to the bit-wise operations on values to apply to pixel maps. In this case by convention, the last operand of the function has the same domain as the result of the function. We start by defining the '*nand*' function.

$$_ \underline{nand} _ : (Pixmap \times Pixmap) \pfun Pixmap$$

$$\begin{aligned}
&\forall MapPair;\ map : Pixmap \bullet \\
&\quad map_1 \underline{nand}\ map_2 = map \Leftrightarrow \\
&\qquad \dom map = \dom map_2 \land \\
&\qquad (\forall pix_1 : \dom map_1;\ pix_2 : \dom map_2 \mid \\
&\qquad\quad (pix_1, map_1) \underline{samepos}\ (pix_2, map_2) \bullet \\
&\qquad\qquad map_1(pix_1)\ \underline{NAND}\ map_2(pix_2) = map\ pix_2\ )
\end{aligned}$$

Note that this operation is not commutative, unlike '*NAND*' since the second operand defines the domain of the result of the function. Specifically, if the domains of the pixel maps differ, the ordering is important. If the domains are the same then the ordering is unimportant.

$$\begin{aligned}
MapPair \vdash\ & \dom map_1 = \dom map_2 \Rightarrow \\
& map_1 \underline{nand}\ map_2 = map_2 \underline{nand}\ map_1
\end{aligned}$$

Using the basic '*nand*' function, we may define fifteen further operations. Four of these degenerate to monadic functions. '*noop*' leaves a pixel map unaffected, '*not*' inverts all bits, '*clear*' clears all the bits and '*set*' sets all the bits.

$$\begin{aligned}
&noop, \\
&not, \\
&clear, \\
&set : Pixmap \to Pixmap
\end{aligned}$$

$$\begin{aligned}
&\forall map : Pixmap \bullet \\
&\quad noop\ map = map \land \\
&\quad not\ map = map \underline{nand}\ map \land \\
&\quad clear\ map = map \underline{nand}\ (not\ map) \land \\
&\quad set\ map = not(clear\ map)
\end{aligned}$$

There are five more important operators. These correspond to standard logical operations except '*copy*' which is extremely useful for moving pixel maps.

$$\begin{aligned}
&_ \underline{and} _, \\
&_ \underline{or} _, \\
&_ \underline{xor} _, \\
&_ \underline{nor} _, \\
&_ \underline{copy} _ : (Pixmap \times Pixmap) \pfun Pixmap
\end{aligned}$$

$$\begin{aligned}
&\forall MapPair \bullet \\
&\quad map_1 \underline{and}\ map_2 = not(map_1 \underline{nand}\ map_2) \land \\
&\quad map_1 \underline{or}\ map_2 = (not\ map_1) \underline{nand}\ (not\ map_2) \land \\
&\quad map_1 \underline{xor}\ map_2 = (map_1 \underline{or}\ map_2) \underline{and}\ (map_1 \underline{nand}\ map_2) \land \\
&\quad map_1 \underline{nor}\ map_2 = not(map_1 \underline{or}\ map_2) \land \\
&\quad map_1 \underline{copy}\ map_2 = map_1 \underline{or}\ (clear\ map_2)
\end{aligned}$$

The rest of the operations are used less often but are detailed here for completeness.

$$
\begin{array}{|l}
_ \underline{copyInverted} _, \\
_ \underline{andReverse} _, \\
_ \underline{andInverted} _, \\
_ \underline{orReverse} _, \\
_ \underline{orInverted} _, \\
_ \underline{equiv} _ : (Pixmap \times Pixmap) \pfun Pixmap \\
\hline
\forall MapPair \bullet \\
\quad map_1 \ \underline{copyInverted} \ map_2 = (not\ map_1) \ \underline{or} \ (clear\ map_2) \land \\
\quad map_1 \ \underline{andReverse} \ map_2 = map_1 \ \underline{and} \ (not\ map_2) \land \\
\quad map_1 \ \underline{andInverted} \ map_2 = (not\ map_1) \ \underline{and} \ map_2 \land \\
\quad map_1 \ \underline{orReverse} \ map_2 = map_1 \ \underline{or} \ (not\ map_2) \land \\
\quad map_1 \ \underline{orInverted} \ map_2 = (not\ map_1) \ \underline{or} \ map_2 \land \\
\quad map_1 \ \underline{equiv} \ map_2 = (not\ map_1) \ \underline{xor} \ map_2
\end{array}
$$

This covers the sixteen possible raster-op Boolean functions on two values.

11.2 Display Operations

A display screen consists of a pixel map.

$$
\begin{array}{|l}
\hline
Display \\
\hline
screen : Pixmap \\
\hline
\end{array}
$$

During changes to the screen, its size does not change although the pixel values displayed on the screen may be updated.

$$
\begin{array}{|l}
\hline
\Delta Display \\
\hline
Display \\
Display' \\
\hline
\dom screen' = \dom screen \\
\hline
\end{array}
$$

The screen may be updated using one of the raster-op functions previously defined. Some functions require a source and destination area while others degenerate into a single area.

$$
\begin{array}{|l}
\hline
RasterOp_1 \\
\hline
\Delta Display \\
area? : \mathbb{P}\, Pixel \\
op? : Pixmap \fun Pixmap \\
\hline
screen' = screen \oplus op?(area? \rhd screen) \\
\hline
\end{array}
$$

RasterOp$_2$

$\Delta Display$
$area? : \mathbb{P}\, Pixel$
$from? : Pixel$
$op? : (Pixmap \times Pixmap) \pfun Pixmap$

$screen' = screen \oplus$
$\quad op?\,(\,((offset\, from?)(\!|\, area? \,|\!)) \lhd screen\,,\, area? \lhd screen\,)$

11.3 An Example – Swapping Pixel Maps

Consider two separate non-overlapping pixel maps with the same shape.

SepPair

$MapPair$

$\neg\, map_1 \;\underline{overlaps}\; map_2$

$\Delta SepPair \mathrel{\widehat=} SepPair \land SepPair'$

We may swap a pair of pixel maps with the same shape using the '*copy*' operation.

CopySwap

$\Delta SepPair$

$map'_1 = map_2 \;\underline{copy}\; map_1$

$map'_2 = map_1 \;\underline{copy}\; map_2$

In practice, these two '*copy*' operations cannot be carried out simultaneously. A third copy is necessary and additionally a temporary pixel map area is required. This can easily be expressed by using three schemas, one for each operation, and then combining them using the schema composition operator ('$\fatsemi$'). This is left as a exercise for the reader.

Alternatively, the '*xor*' raster-op function may be used. Three sequential operations are still necessary, but the use of a temporary buffer area is eliminated. The following two schemas '*xor*' one or other of a pair of pixel maps with its opposite number.

Xor$_1$

$\Delta SepPair$

$map'_1 = map_2 \;\underline{xor}\; map_1$

$map'_2 = map_2$

Xor$_2$

$\Delta SepPair$

$map'_1 = map_1$

$map'_2 = map_1 \;\underline{xor}\; map_2$

A swap may be achieved between the two pixel maps by applying '*xor*' three times sequentially as follows.

$$XorSwap \mathrel{\widehat=} Xor_1 \fatsemi Xor_2 \fatsemi Xor_1$$

By symmetry, the following is also true.

$$\Delta SepPair \vdash XorSwap \Leftrightarrow (Xor_2 \fatsemi Xor_1 \fatsemi Xor_2)$$

The *XorSwap* operation has exactly the same effect as using the '*copy*' operator twice simultaneously, as in the *CopySwap* operation.

$$\Delta SepPair \vdash CopySwap \Leftrightarrow XorSwap$$

11.4 Conclusion

In this chapter we have formally specified the raster-op function and how this may be applied to a graphics display. We have given an example of its use for swapping areas of a display. By defining operations performed on rectangular areas, we have specified some of the underlying operations necessary for a window system. Three different window systems are formalized in Part VI.